



Engenharia de Software e Aplicações

- Aula 6 – Desenvolvimento ágil de software

- Prof. Esp.: Maurício das Neves
- Segunda-Feira : 19:00 as 22:30 hrs

Engenharia de Softwares 1

Objetivos:

Na aula de hoje vamos conhecer o desenvolvimento ágil de software, você vai compreender:

A lógica do desenvolvimento ágil de software, o manifesto ágil e as diferenças entre o desenvolvimento ágil e o dirigido por plano;

Conhecerá as práticas importantes do desenvolvimento ágil, como as histórias do usuário, a refatoração, a programação em pares e o desenvolvimento com testes a priori (test-first);

Compreenderá desde a abordagem Scrum até o gerenciamento ágil de projetos;

Compreenderá as questões de escalabilidade dos métodos de desenvolvimento ágil e a combinação das abordagens ágeis com as dirigidas por plano no desenvolvimento de grandes sistemas de software.

Engenharia de Softwares 1

As empresas de hoje em dia operam em um ambiente global em rápida mudança. Elas precisam responder às novas oportunidades e mercados, às mudanças nas condições econômicas e ao surgimento de produtos e serviços concorrentes. O software faz parte de quase todas as operações de negócios, então novo software tem de ser desenvolvido rapidamente, para que seja possível tirar vantagens das novas oportunidades e responder a pressão da concorrência. **A entrega e desenvolvimento rápidos** são, portanto, requisitos mais importantes da maioria dos sistemas de negócios. Na verdade, **as empresas podem estar dispostas a negociar a qualidade e comprometer requisitos se puderem implantar um novo software rapidamente.**

Como essas empresas operam em um **ambiente dinâmico**, é praticamente impossível derivar um conjunto completo de requisitos de software estáveis. Esses requisitos mudam porque **os clientes acham impossível prever** como um sistema irá afetar as práticas profissionais, como irá interagir com outros

Engenharia de Softwares 1

sistemas e quais operações de usuário devem ser automatizadas. Pode ser que só depois que um sistema tenha sido entregue, e os usuários tenham adquirido experiência com ele, que os verdadeiros requisitos fiquem claros. Mesmo assim fatores externos orientam sua mudança.

Os processos de desenvolvimento de software dirigidos por plano que especificam completamente os requisitos e depois projetam, constroem e testam um sistema não são voltados para o desenvolvimento rápido de software. A medida em que os requisitos mudam ou que problemas de requisitos são descobertos, o projeto ou a implementação do sistema precisam ser retrabalhados e testados novamente. Como consequência, um projeto convencional em cascata ou baseado em especificação normalmente é demorado e o software final é entregue ao cliente muito depois do prazo originalmente estipulado.

Engenharia de Softwares 1

Para alguns tipos de software, como sistemas de controle críticos em segurança, para os quais uma análise completa do sistema é essencial, essa abordagem dirigida por plano é mais indicada. No entanto, em um ambiente empresarial dinâmico, isso pode ser problemático. Quando o software finalmente estiver disponível para uso, o motivo original de sua aquisição pode ter mudado tão radicalmente que ele acaba sendo INÚTIL. Portanto, especialmente no caso de sistemas de negócio, os processos de desenvolvimento e entrega RÁPIDOS são essenciais.

A necessidade de desenvolvimento rápido de software e de processos que possam lidar com requisitos que mudam foi reconhecida há muitos anos. Entretanto, o desenvolvimento mais rápido do software só decolou no final dos anos 90, quando surgiu a ideia de métodos ágeis como a Programação Extrema, do inglês Extreme Programming, ou XP, o Scrum e o DSM, do inglês Dynamic Systems Development Method.

Engenharia de Softwares 1

O **desenvolvimento rápido de software** passou a ser conhecido como **desenvolvimento ágil, ou métodos ágeis**. Esses métodos são concebidos para produzir software útil de maneira rápida. Todos eles compartilham uma série de características comuns:

1. Os processos de especificação, projeto e implementação são intercalados. Não há especificação detalhada do sistema e a documentação do projeto é minimizada ou gerada automaticamente pelo ambiente de programação utilizado para implementar o sistema. O documento de requisitos do usuário é uma definição resumida contendo apenas as características mais importantes do sistema.
2. O sistema é desenvolvido em uma série de incrementos. Os usuários finais estão envolvidos na especificação e avaliação de cada um deles. Além de mudanças no software, eles também podem propor novos requisitos para serem implementados em uma versão posterior do sistema.

Engenharia de Softwares 1

3. O amplo apoio de ferramentas é usado para ajudar no processo de desenvolvimento. Podem ser utilizadas ferramentas de teste automatizado, ferramentas de apoio ao gerenciamento de configuração e à integração de sistemas, além de ferramentas para automatizar a produção da interface com o usuário.

Os métodos ágeis se baseiam no desenvolvimento incremental: os incrementos são pequenos e, normalmente, novas versões do sistema são criadas e disponibilizadas para os clientes a cada duas ou três semanas, para que seja possível obter deles um feedback rápido nos requisitos que mudam. Além disso, esse método minimiza a documentação, usando a comunicação informal em vez de reuniões formais com documentos escritos.

Engenharia de Softwares 1

As abordagens ágeis de desenvolvimento de software consideram o projeto (design) e a implementação como as atividades centrais no processo de software. Elas incorporam outras tarefas a essas atividades, **como a elicitação dos requisitos e os testes**. Por outro lado, uma abordagem dirigida por plano identifica etapas diferentes no processo de software, com resultados associados a cada uma delas. Esses dados são utilizados como base para o planejamento da atividade de processo seguinte.

A figura abaixo mostra as diferenças fundamentais entre as abordagens ágil e dirigida por plano na especificação de sistemas. Em um processo de desenvolvimento de software dirigido por plano, a iteração ocorre dentro das atividades, com documentos formais sendo utilizados como meio de comunicação entre as etapas do processo. Por exemplo: os requisitos evoluirão e, no final das contas, será produzida uma especificação deles, que servirá como entrada para o processo de projeto e implementação.

Engenharia de Softwares 1

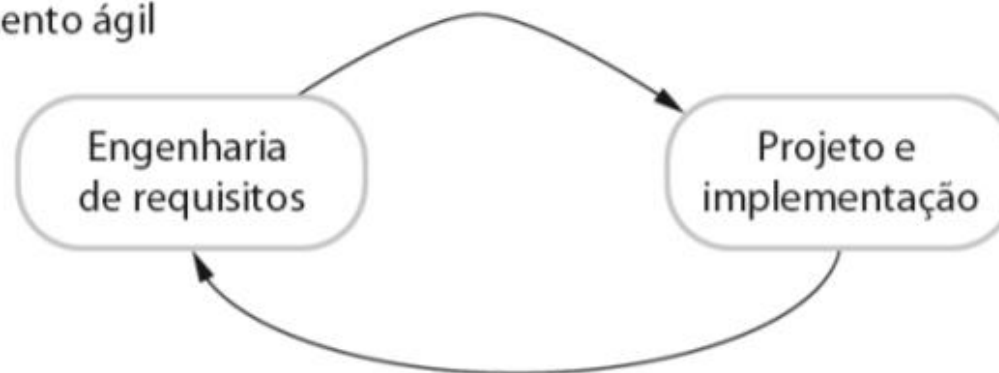
Em uma abordagem ágil, por outro lado, a iteração ocorre ao longo das atividades.

Portanto, os requisitos e o projeto (design) são desenvolvidos juntos, e não separadamente.

Desenvolvimento baseado em planos



Desenvolvimento ágil



Engenharia de Softwares 1

Na prática, conforme veremos em outras aulas, os processos dirigidos por plano são utilizados frequentemente com práticas de programação ágil, e os métodos ágeis podem incorporar algumas atividades planejadas, além da programação e dos testes.

Em um processo dirigido por plano, é perfeitamente viável alocar requisitos e planejar a fase de projeto (design) e desenvolvimento como uma série de incrementos. Já um processo ágil não é inevitavelmente focado no código e pode produzir alguma documentação de projeto (design). Nos métodos ágeis, os desenvolvedores podem decidir que uma iteração não produzirá código novo, mas sim modelos e documentação de sistema.

Engenharia de Softwares 1

Métodos ágeis

Nos anos 1980 e início dos anos 1990, havia uma percepção generalizada de que a maneira mais indicada para obter um software melhor era por meio do planejamento cuidadoso do projeto, da garantia de qualidade formalizada, do uso de métodos de análise e projeto (design) apoiados por ferramentas de software e de processos de desenvolvimento controlados e rigorosos. Essa percepção veio da comunidade de engenharia de software que era responsável por desenvolver sistemas grandes e duradouros, como os destinados aos setores aeroespacial e governamental.

Essa abordagem dirigida por plano foi desenvolvida para o software criado por grandes times, trabalhando para diferentes empresas. Os times costumavam ficar dispersos geograficamente e trabalhavam no software por longos períodos. Um exemplo de software produzido dessa maneira são os sistemas de controle de uma aeronave moderna que levavam até 10 anos para serem

Engenharia de Softwares 1

desenvolvidos desde sua especificação inicial até sua implementação. As abordagens dirigidas por plano envolvem sobrecargas no planejamento, no desenvolvimento e na documentação do sistema. Essa sobrecarga é justificada quando o trabalho de vários times de desenvolvimento precisa ser coordenado, quando o sistema é crítico e quando muitas pessoas diferentes estarão envolvidas na manutenção do software ao longo de sua vida útil.

No entanto, quando essa abordagem pesada é aplicada a sistemas de negócio de **pequeno ou médio porte**, a sobrecarga é tão grande que domina o processo de desenvolvimento de software. Mais tempo é investido na decisão de como o sistema deve ser desenvolvido do que na programação ou nos testes. À medida que os requisitos mudam, retrabalho é necessário e, ao menos a princípio, sua especificação e seu projeto (design) tem de mudar.

Engenharia de Softwares 1

A insatisfação com essas abordagens levou ao desenvolvimento dos métodos ágeis no final da década de 1990. Eles permitiram que o time de desenvolvimento se concentrasse no próprio software, em vez de no projeto (design) ou na documentação. Os **métodos ágeis são mais adequados** para desenvolver aplicações nas quais os **requisitos dos sistema mudam rapidamente durante o processo**. Eles se destinam a fornecer rapidamente um software funcional para o cliente, que, por sua vez, pode propor a inclusão de requisitos novos ou modificados nas iterações seguintes. **Eles visam reduzir a burocracia do processo** ao evitar o trabalho com valor duvidoso no longo prazo e eliminar a documentação que provavelmente jamais será utilizada.

A filosofia por trás dos métodos ágeis está refletida no “manifesto ágil” (<http://agilemanifesto.org>), criados por desenvolvedores líderes desses métodos.

Engenharia de Softwares 1

O manifesto diz:

Estamos descobrindo maneiras melhores de desenvolver software, fazendo-o nós mesmos e ajudando outros a fazerem o mesmo. Através deste trabalho passamos a valorizar:

Indivíduos e interações mais que processos e ferramentas.

Software em funcionamento mais que uma documentação abrangente.

Colaboração com o cliente mais que negociação de contratos.

Responder a mudanças mais que seguir um plano.

Ou seja, mesmo havendo valor nos itens à direita, valorizamos mais os itens à esquerda.

Engenharia de Softwares 1

Todos os métodos ágeis sugerem que o software deve ser desenvolvido e entregue incrementalmente. Esses métodos se baseiam em diferentes processos ágeis, mas compartilham um conjunto de princípios, com base no manifesto ágil, e por isso tem muito em comum.

Princípios	Descrição
Envolvimento do cliente	Os clientes devem estar intimamente envolvidos no processo de desenvolvimento. Seu papel é fornecer e priorizar novos requisitos do sistema e avaliar suas iterações.
Entrega incremental	O software é desenvolvido em incrementos com o cliente, especificando os requisitos para serem incluídos em cada um.
Pessoas, não processos	As habilidades da equipe de desenvolvimento devem ser reconhecidas e exploradas. Membros da equipe devem desenvolver suas próprias maneiras de trabalhar, sem processos prescritivos.
Aceitar as mudanças	Deve-se ter em mente que os requisitos do sistema vão mudar. Por isso, projete o sistema de maneira a acomodar essas mudanças.
Manter a simplicidade	Focalize a simplicidade, tanto do software a ser desenvolvido quanto do processo de desenvolvimento. Sempre que possível, trabalhe ativamente para eliminar a complexidade do sistema.

Engenharia de Softwares 1

Os métodos ágeis tem sido particularmente úteis para dois tipos de desenvolvimento de sistemas:

1. O desenvolvimento de um produto pequeno ou médio, por uma empresa de software, para venda. Praticamente todos os produtos de software e aplicativos são desenvolvidos atualmente usando uma abordagem ágil.
2. O desenvolvimento de sistemas personalizados dentro de uma organização, em que há um compromisso claro por parte do cliente de se **envolver no processo de desenvolvimento**, e no qual há poucas stakeholders externos e normas que afetem o software.

Engenharia de Softwares 1

Os métodos ágeis funcionam bem nessas situações porque possibilitam comunicação contínua entre **gerente de produto** ou o **cliente do sistema** e o **time de desenvolvimento**. O software em si é um sistema stand-alone, em vez de estar intimamente integrado com outros sistemas que estejam sendo desenvolvidos simultaneamente. Consequentemente, não há necessidade de coordenar fluxos de desenvolvimento paralelos. Os sistemas pequenos e médios podem ser desenvolvidos por times situados no mesmo local, então a comunicação informal entre seus membros funciona bem.

Engenharia de Softwares 1

Técnicas de Desenvolvimento Ágil

Primeiramente, precisamos entender a diferença entre o método ágil e a abordagem tradicional. O método tradicional entende que o produto deve ser entregue ao cliente e avaliado em sua totalidade.

O desenvolvimento ágil aposta em um **diálogo constante entre o desenvolvedor e o cliente**, o que prevê **avaliações permanentes e correções ainda durante o processo**. O objetivo é oferecer uma solução completa que atenda de fato às demandas do usuário.

Engenharia de Softwares 1

Agora vamos conhecer o desenvolvimento ágil de software, você vai compreender:

A lógica do desenvolvimento ágil de software, o manifesto ágil e as diferenças entre o desenvolvimento ágil e o dirigido por plano;

Conhecerá as práticas importantes do desenvolvimento ágil, como as histórias do usuário, a refatoração, a programação em pares e o desenvolvimento com testes a priori (test-first);

Compreenderá desde a abordagem Scrum até o gerenciamento ágil de projetos;

Compreenderá as questões de escalabilidade dos métodos de desenvolvimento ágil e a combinação das abordagens ágeis com as dirigidas por plano no desenvolvimento de grandes sistemas de software.

Engenharia de Softwares e Aplicações

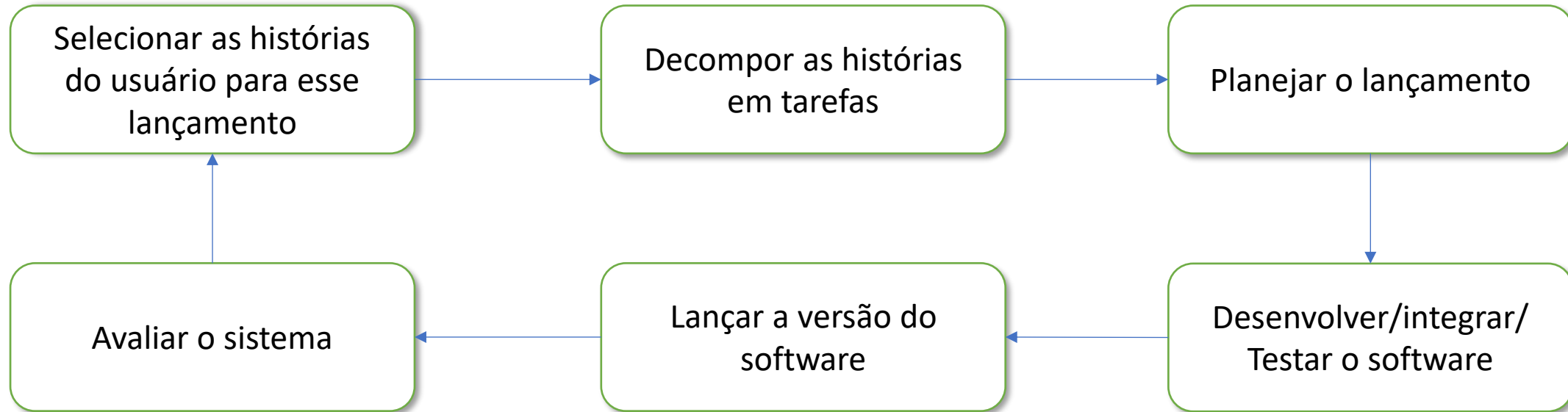
Técnicas de Desenvolvimento Ágil

As ideias por trás dos métodos ágeis foram desenvolvidas mais ou menos na mesma época, nos anos 1990, por uma série de pessoas. Entretanto, talvez, a abordagem mais importante para mudança de cultura no desenvolvimento de software tenha sido o desenvolvimento de Programação Extrema (do inglês Extreme Programming, ou XP). O nome foi cunhado por Kent Beck com base na ideia de que essa abordagem leva a níveis “extremos” boas práticas reconhecidas, como o desenvolvimento iterativo. Na programação extrema, por exemplo, várias versões novas de um sistema podem ser desenvolvidas – por diferentes programadores -, integradas e testadas em um dia.

A figura adiante ilustra o processo de produção do incremento de um sistema que está sendo desenvolvido usando Programação Extrema.

Engenharia de Softwares 1

Ciclo de lançamento (release) da Programação Extrema



Engenharia de Softwares 1

Técnicas de Desenvolvimento Ágil

Na XP, os requisitos são expressos em cenários (**chamados de história do usuário**) implementados diretamente como uma série de tarefas. Os programadores trabalham **em pares** e desenvolvem **testes para cada tarefa** antes de escreverem o código. Todos os testes devem ser executados com sucesso quando o novo código é integrado ao sistema, já que há um curto intervalo de tempo entre os lançamentos (release) do sistema.

A Programação Extrema era controversa, pois introduziu uma série de práticas ágeis muito diferentes do desenvolvimento tradicional da época. Elas estão resumidas mais adiante e refletem os princípios do manifesto ágil:

1. O desenvolvimento incremental é apoiado por lançamentos menores e mais frequentes do sistema. Os requisitos se baseiam em **histórias simples dos clientes**, ou cenários, utilizados com base para decidir qual funcionalidade deve ser incluída em um determinado incremento.

Engenharia de Softwares 1

Técnicas de Desenvolvimento Ágil

2. O envolvimento do cliente é apoiado por seu engajamento contínuo no time de desenvolvimento. O **representante do cliente participa do desenvolvimento** e é o responsável por definir os testes de aceitação do sistema.
3. As pessoas, não os processos, são apoiadas pela **programação em pares**, pela propriedade coletiva do código do sistema e por um processo de desenvolvimento sustentável que não envolve expedientes de trabalho longos demais.
4. As mudanças são adotadas por meio de **lançamentos regulares** do sistema aos clientes, desenvolvimento com **testes a priori (test-first)**, **refatoração** para evitar a degeneração do código e **integração contínua de novas funcionalidades**.

Engenharia de Softwares 1

Técnicas de Desenvolvimento Ágil

5. A manutenção da simplicidade é apoiada pela **refatoração constante**, que melhora a qualidade do código, e pelo uso de projetos (design) simples, **que não antecipam desnecessariamente** as futuras mudanças no sistema.

Na prática, a aplicação da Programação Extrema como proposta originalmente se provou mais difícil do que o previsto. Ela não pode ser integrada de imediato às práticas de gestão e à cultura da maioria das empresas; assim, as que adotam métodos ágeis selecionam as práticas de Programação Extrema mais adequadas ao seu modo de trabalho. Às vezes, essas técnicas são incorporadas aos processos de desenvolvimento das próprias empresas, mas, na maioria das vezes, são utilizadas em conjunto com um método ágil focado em gerenciamento, como o Scrum.

Engenharia de Softwares 1

Práticas de Programação Extrema

Princípio ou prática	Descrição
Propriedade Coletiva	Os pares de desenvolvedores trabalham em todas as áreas do sistema de modo que não se desenvolvem “ilhas de conhecimento”, e todos os desenvolvedores assumem responsabilidade por todo o código. Qualquer um pode mudar qualquer coisa
Integração contínua	Assim que o trabalho em uma tarefa é concluído, ele é integrado ao sistema completo. Após qualquer integração desse tipo, todos os testes de unidade no sistema devem passar.
Planejamento incremental	Os requisitos são registrados em “cartões de história”, e as histórias a serem incluídas em um lançamento são determinadas de acordo com o tempo disponível e com sua prioridade relativa. Os desenvolvedores decompõe essas histórias em tarefas de desenvolvimento.
Representante do cliente	Um representante do usuário final do sistema (cliente) deve estar disponível em tempo integral para o time de programação. Em um processo como esse, o cliente é um membro do time de desenvolvimento, sendo responsável por levar os requisitos do sistema ao time, visando sua implementação.
Programação em pares	Os desenvolvedores trabalham em pares, conferindo o trabalho um do outro e oferecendo o apoio necessário para que o resultado final seja sempre satisfatório.

Engenharia de Softwares 1

Práticas de Programação Extrema

Princípio ou prática	Descrição
Refatoração	Todos os desenvolvedores devem refatorar o código continuamente logo que sejam encontradas possíveis melhorias para ele. Isso mantém o código simples e de fácil manutenção.
Projeto simples	Deve ser feito o suficiente de projeto (design) para satisfazer os requisitos atuais, e nada mais.
Lançamentos pequenos	O mínimo conjunto útil de funcionalidade que agregue valor ao negócio é desenvolvido em primeiro lugar. Os lançamentos do sistema são frequentes e acrescentam funcionalidade à primeira versão de uma maneira incremental.
Ritmo sustentável	Grandes quantidade de horas extras não são consideradas aceitáveis, já que o efeito líquido muitas vezes é a diminuição da qualidade do código e da produtividade no médio prazo.
Desenvolvimento com testes a priori (test-first)	Um framework automatizado de teste de unidade é utilizado para escrever os testes de um novo pedaço de funcionalidade antes que ela própria seja implementada.

Não estou convencido de que a Programação Extrema seja um método ágil prático para a maioria das empresas, mas sua contribuição mais importante é, provavelmente o conjunto de práticas de desenvolvimento ágil que introduziu na comunidade.

Engenharia de Softwares 1

Histórias do Usuário

Os requisitos sempre mudam. Para lidar com essas mudanças, os métodos ágeis não tem um atividade de engenharia de requisitos específica ou independente. Em vez disso, a elicitação dos requisitos é integrada ao desenvolvimento. Para facilitar esse processo, foi desenvolvida a ideia de “histórias do usuário”, com o intuito de formar cenários de uso baseados nas experiências de um usuário do sistema.

Na medida do possível, o cliente do sistema trabalha em estreita colaboração com o time de desenvolvimento e discute esses cenários com os membros do time. Juntos, eles desenvolvem um “cartão de história” que descreve resumidamente uma história que reúna as necessidades do usuário.

O time de desenvolvimento, buscará, então, implementar esse cenário em uma versão futura do software.

Engenharia de Softwares 1

Histórias do Usuário

Um exemplo do cartão de história do sistema base do nosso autor (Mentcare), será mostrado mais abaixo. Trata-se de uma descrição sucinta de um cenário para prescrever medicação para um paciente.

Prescrição de medicação

Kate é uma médica que deseja prescrever medicação para um paciente que frequenta sua clínica. O registro do paciente já é exibido em seu computador, então ela deve clicar no campo <Medicação> e selecionar <Medicação Atual>, <Nova Medicação> ou <Substâncias>.

Se escolher <Medicação Atual>, o sistema pedirá para que ela verifique a dose; se quiser mudá-la, Kate fornecerá a nova dose e depois deverá confirmar a prescrição.

Se escolher <Nova Medicação>, o sistema presumirá que ela sabe qual medicação prescrever. Ao digitar as primeiras letras da medicação, o sistema exibirá uma lista de possíveis medicamentos que começam com essas letras.

Engenharia de Softwares 1

Histórias do Usuário

Prescrição de medicação

Ao escolher a medicação necessária, o sistema responderá pedindo que ela confirme se a medicação escolhida está correta. Ela deverá fornecer a dose e depois confirmar a prescrição.

Se ela escolher <Substâncias>, o sistema exibirá uma caixa de busca para a substância aprovada, e então ela poderá pesquisar o medicamento que deseja. Feita a busca lhe será solicitado que confirme que o medicamento selecionado está correto. Ela deverá fornecer a dose e depois confirmar a prescrição.

O sistema sempre verificará se a dose está dentro da faixa aprovada. Se não estiver será pedido a Kate que faça uma alteração.

Depois de confirmar a prescrição, ela será exibida para conferência. Kate deverá clicar em “OK” ou em “Alterar”. Se “OK” for selecionado a prescrição será registrada no banco de dados de auditoria. Se clicar em “Alterar”, o processo de “Prescrição de Medicação” é executado novamente.

Engenharia de Softwares 1

Histórias do Usuário

As histórias do usuário podem ser utilizadas no planejamento das iterações do sistema. Depois de desenvolvidos, os cartões de história devem ser decompostos em tarefas pelo time de desenvolvimento, **imagem adiante**, que estima o esforço e os recursos necessários para implementar cada uma delas. Normalmente, isso envolve discussões com o cliente para refinar os requisitos.

O cliente prioriza as histórias a serem implementadas, escolhendo as que podem ser usadas imediatamente para proporcionar suporte útil ao negócio. A intenção é identificar funcionalidades essenciais que possam ser implementadas em aproximadamente duas semanas, quando a próxima versão do sistema é disponibilizada para o cliente.

Engenharia de Softwares 1

Exemplo de cartões de tarefa

Tarefa 1: Mudar a dose do medicamento prescrito

Tarefa 2: Seleção de Substância

Tarefa 3: Verificação da dose

A verificação da dose é uma precaução de segurança para conferir se o médico não prescreve uma dose perigosamente pequena ou grande.

Usando a identificação da substância para o nome genérico do medicamento, procure a substância e recupere a dose máxima e a mínima recomendadas.

Confira a dose prescrita em relação ao máximo e ao mínimo. Se estiver fora da faixa, emita uma mensagem de erro dizendo que a dose é alta ou baixa demais. Se estiver dentro da faixa, habilite o botão <Confirmar>.

Engenharia de Softwares 1

Naturalmente, **a medida que os requisitos mudam**, as histórias não implementadas **também mudam ou são descartadas**. Se forem necessárias alterações em um sistema **que já foi entregue**, são elaborados **novos cartões de história** e, mais uma vez, **o cliente decide se essas alterações devem ter ou não prioridade** sobre novas funcionalidades.

A ideia de histórias do usuário é poderosa, e as pessoas **acham muito mais fácil se identificar com elas do que com o documento convencional de requisitos ou casos de uso**. As histórias podem ser úteis para fazer com que os usuários se **envolvam na sugestão de requisitos** durante uma atividade de elicitação que anteceda o desenvolvimento.

Por outro lado, **o problema principal com as histórias do usuário é a completude**. É difícil julgar se foram desenvolvidas histórias suficientes para cobrir todos os requisitos essenciais de um sistema.

Engenharia de Softwares 1

Também é difícil jogar se uma única história proporciona uma imagem completa de uma atividade. **Usuários mais experientes estão frequentemente tão familiarizados com seu trabalho que deixam de fora algumas coisas quando o descrevem.**

Depois de escrever uma história, ela deve ser integrada ao fluxo de trabalho. **Em geral, a história é escrita pelo proprietário do produto, gerente de produto ou gerente de programa** e enviada para revisão.

Durante uma reunião de planejamento de sprint ou iteração, **a equipe decide quais histórias vão ser trabalhadas nesse sprint.** Então, as equipes discutem os requisitos e a funcionalidade que cada história de usuário requer. Esta é uma oportunidade para a equipe ser técnica e criativa na implementação da história. Assim que forem combinados, esses requisitos são adicionados à história.

Engenharia de Softwares 1

Considerações.